

CONTROLADOR DE JUEGO BLE HID PARA RASPBERRY PI ZERO 2W CON PICO 8

Elaborado por: Santiago Galeano Castaño

Juan Diego Restrepo Hernández

Presentado a:

Profesor Juan Bernardo Gómez

Asignatura:

Sistemas Linux Embebidos

UNIVERSIDAD NACIONAL DE COLOMBIA
Sede Manizales

Diciembre 6 de 2025

Índice

Índice General2Lista de Figuras3

1. Introducción y Marco Teórico	1
Resumen (Abstract)	1
2. Motivación y Justificación	1
3. Objetivos del Proyecto	1
3.1. Objetivo General	1
3.2. Objetivos Específicos	1
4. Estructura del Documento	2
5. Fundamentos Teóricos y Estado del Arte	2
5.1. Microcontrolador ESP32	2
5.2. Protocolo Bluetooth Low Energy (BLE)	3
5.2.1. Arquitectura GATT	3
5.2.2. Concepto de Bonding	3
5.3. Estándar Human Interface Device (HID)	3
5.3.1. Report Descriptor	3
5.3.2. Formato del Reporte de Entrada	3
5.4. Estado del Arte	4
6. Arquitectura y Diseño del Sistema	4
6.1. Diseño de Hardware	4
6.1.1. Diagrama de Bloques Físicos	4
6.1.2. Selección de Componentes e Interfaz	4
6.2. Diseño de Software (Firmware)	4
6.2.1. Arquitectura Modular de Software	5
6.2.2. Diagrama de Flujo del Firmware	5
6.3. Procesamiento de Datos en Tiempo Real	5
6.4. Diagrama Completo del Sistema	5
Diagramas Arquitectónicos Clave	5
Diagrama de Bloques del Hardware (Gamepad)	6
Diagrama de Secuencia de Comunicación BLE HID	6
Diagrama General de la Lógica del Juego	6
7. Implementación y Desarrollo	6
7.1. Procesamiento de la Señal del Joystick	6
7.1.1. Normalización y Calibración	6
7.1.2. Aplicación de la Zona Muerta (Dead Zone)	6
7.1.3. Conversión a Reporte HID	7
7.2. Implementación del Perfil Bluetooth HID (HOGP)	7
7.2.1. El Report Descriptor	7
7.2.2. Mecanismo de Transmisión	7
7.3. Integración FreeRTOS y Control de Flujo	7

8. Configuración de la Plataforma Destino	8
8.1. Requisitos del Sistema Operativo e Instalación de Dependencias	8
8.2. Transferencia y Preparación de PICO-8	8
8.3. Arranque Automático en Modo Kiosk	8
8.4. Solución de Compatibilidad de Arquitectura (ARMv7/ARMv8)	9
9. Pruebas, Resultados y Análisis	9
9.1. Metodología de Verificación	9
9.1.1. Pruebas Unitarias del Firmware	9
9.1.2. Pruebas Funcionales de Integración (End-to-End)	10
9.2. Análisis de Rendimiento y Estabilidad	10
9.2.1. Latencia del Sistema	10
9.2.2. Estabilidad de la Comunicación	11
9.3. Solución de Problemas y Observaciones	11
9.4. Consumo de Energía	11
10.Descripción del Juego	12
10.1. Mecánicas Principales	12
10.2. Controles del Controlador BLE	12
11.Componentes Técnicos Clave del Juego	12
11.1. Sistema de Estados y Animación	12
11.2. Sistema de Reconocimiento de Gestos	12
11.3. Dificultad Progresiva y Optimizaciones	13
12.Mecánica y Controles de Validación	13
13.Diagramas de Flujo de la Lógica del Juego	13
14.Diagramas de Algoritmos Clave del Juego	13
14.1. Diagrama General de la Lógica del Juego	13
14.2. Diagrama del Algoritmo de Detección de Gestos	13
14.3. Diagrama de la Generación y Movimiento de Enemigos	13
15.Conclusiones y Trabajo Futuro	14
15.1. Conclusiones	14
15.2. Trabajo Futuro	14

Índice de figuras

1. Diagrama de Bloques del Hardware del Controlador.	4
2. Diagrama de Flujo de la Tarea Principal del Gamepad.	5
3. Medición de Latencia End-to-End (Prueba 1). La distancia horizontal entre el flanco de bajada (amarillo) y el inicio del cambio (cian) representa la latencia total del sistema.	11
4. Medición de Latencia End-to-End (Prueba 2). Confirmación de la medición de latencia total.	11

1. Introducción y Marco Teórico

Resumen (Abstract)

Resumen

Este documento presenta el diseño e implementación de un Controlador de Juego (Gamepad) de Fabricación Propia utilizando el microcontrolador ESP32 para establecer una conexión inalámbrica mediante el perfil Bluetooth Low Energy. El proyecto aborda la necesidad de un dispositivo de entrada de baja latencia y bajo consumo, diseñado específicamente para la plataforma PICO-8 en la Raspberry Pi Zero 2W. El sistema implementa la lectura continua de un joystick analógico de dos ejes y cuatro botones digitales, aplicando técnicas de procesamiento digital de señales (calibración, zona muerta y curva de respuesta) para generar un reporte HID estándar de Gamepad. Se detalla la arquitectura de firmware basado en ESP-IDF y FreeRTOS, el proceso de bonding, y la configuración de la Raspberry Pi en modo kiosk para la ejecución automática de PICO-8. Los resultados validan una solución de juego inalámbrica, estable y optimizada para sistemas Linux embebidos.

2. Motivación y Justificación

La popularidad de las plataformas de emulación y los juegos retro en dispositivos embebidos como la Raspberry Pi Zero 2W ha generado una creciente demanda de soluciones de entrada inalámbricas y de bajo consumo. Si bien existen controladores comerciales, la implementación de una solución permite una integración más profunda y la personalización del firmware para optimizar el rendimiento.

El proyecto se justifica en los siguientes

puntos:

- **Portabilidad y Diseño a Medida:** Creación de un controlador con dimensiones y características físicas adaptadas específicamente para la Zero 2W y el software PICO-8.
- **Optimización de Latencia y Consumo:** Control total sobre la lógica de muestreo y transmisión del reporte HID, permitiendo equilibrar la tasa de refresco (20 Hz) con la eficiencia energética que requiere el protocolo BLE.
- **Experiencia Educativa:** Sirve como caso de estudio práctico para la implementación de protocolos de comunicación complejos (BLE HID) en un sistema embedded usando el framework ESP-IDF.

3. Objetivos del Proyecto

El proyecto se guía por un objetivo general y varios objetivos específicos y medibles que deben cumplirse para considerarse exitoso.

3.1. Objetivo General

Diseñar e implementar un controlador de juego inalámbrico basado en el microcontrolador ESP32 que utilice el perfil Bluetooth Low Energy (BLE) HID para ser reconocido automáticamente como un dispositivo de entrada estándar por la Raspberry Pi Zero 2W.

3.2. Objetivos Específicos

- Desarrollar el firmware en ESP-IDF y FreeRTOS para la lectura de cuatro botones digitales y un joystick analógico de dos ejes.

- Implementar un módulo de procesamiento de señal que incluya calibración, normalización y la aplicación de una zona muerta (Dead Zone) del 12 % a los datos del joystick.
- Configurar el stack de comunicación del ESP32 como un dispositivo BLE HID Gamepad compatible con el perfil GATT, con una frecuencia de reporte de datos de al menos 20 Hz (cada 50 ms).
- Integrar y probar el controlador con la plataforma PICO-8 ejecutándose automáticamente en una Raspberry Pi Zero 2W en modo kiosk.

4. Estructura del Documento

Este informe se encuentra organizado de la siguiente manera:

- **Capítulo 2: Fundamentos Teóricos.** Cubre los conceptos de BLE, el Perfil HID y el microcontrolador ESP32.
- **Capítulo 3: Arquitectura y Diseño del Sistema.** Presenta los diagramas de bloques de hardware y software, y el diseño modular del firmware.
- **Capítulo 4: Implementación y Desarrollo.** Detalla la lógica de programación para la lectura de hardware, el procesamiento de señales y la transmisión BLE.
- **Capítulo 5: Configuración de la Plataforma Destino.** Describe el proceso para la instalación y configuración automática de PICO-8 en la Raspberry Pi.
- **Capítulo 6: Pruebas, Resultados y Análisis.** Valida el funcionamiento del controlador frente a los objetivos específicos y discute las métricas de rendimiento.
- **Capítulo 7: Conclusiones y Trabajo Futuro.** Resume los hallazgos y propone mejoras al proyecto.

5. Fundamentos Teóricos y Estado del Arte

5.1. Microcontrolador ESP32

El ESP32 es un sistema en chip (SoC) desarrollado por Espressif Systems que integra Wi-Fi y Bluetooth de doble modo. Es el componente central del controlador debido a su capacidad para operar como un dispositivo Bluetooth de bajo consumo (BLE) y su arquitectura de doble núcleo, que facilita la ejecución de tareas concurrentes de tiempo real, como la lectura de entradas y la gestión del protocolo de comunicación.

Las características clave aprovechadas en este proyecto son:

- **Bluetooth Low Energy (BLE):** Esencial para implementar el perfil HID con un consumo mínimo de energía.
- **Periféricos de Entrada/Salida (GPIO):** Utilizados para leer el estado de los botones digitales (A, B, SELECT, START).
- **Convertidor Analógico-Digital (ADC):** Necesario para muestrear las señales analógicas generadas por los ejes X e Y del joystick.
- **FreeRTOS:** El sistema operativo en tiempo real (RTOS) subyacente que permite organizar la lógica del firmware en tareas especializadas (ej. la tarea principal del gamepad) para garantizar la regularidad y el determinismo en el muestreo de datos.

5.2. Protocolo Bluetooth Low Energy (BLE)

BLE es un protocolo de red inalámbrica personal diseñado para proporcionar conectividad con un consumo energético muy bajo, ideal para dispositivos que funcionan con baterías. La comunicación en BLE se organiza en perfiles y servicios que definen la estructura y el significado de los datos intercambiados.

5.2.1. Arquitectura GATT

El protocolo BLE se basa en el Generic Attribute Profile (GATT). Este perfil define la forma en que se organizan los datos:

- **Servicios:** Colecciones de datos relacionados. En este proyecto, se utiliza el Servicio de Dispositivo de Interfaz Humana (HID Service).
- **Características:** Unidades de datos dentro de un servicio. Cada característica consta de un valor y propiedades (lectura, escritura, notificación) que definen cómo se accede a dicho valor.
- **Descriptors (Descriptores):** Metadatos que describen el valor de una característica (ej. formato o unidad).

El controlador ESP32 opera como un Servidor GATT, poniendo a disposición del cliente (la Raspberry Pi) las características del servicio HID.

5.2.2. Concepto de Bonding

El bonding es el proceso de creación de una clave de enlace (link key) persistente entre dos dispositivos BLE. Este proceso se realiza una sola vez para que los dispositivos puedan volver a conectarse automáticamente en el futuro sin necesidad de pasar por el proceso de emparejamiento completo cada vez. En un dispositivo de entrada, esto es crucial para la facilidad de uso.

5.3. Estándar Human Interface Device (HID)

El protocolo HID es un estándar que permite a los hosts (ordenadores, Raspberry Pi) comunicarse con dispositivos de entrada y salida humanos (teclados, ratones, gamepads). En el contexto de BLE, se conoce como HID sobre GATT (HOGP).

5.3.1. Report Descriptor

El Report Descriptor es la parte más importante del estándar HID. Es un bloque de datos que el dispositivo (ESP32) envía al host (Raspberry Pi) durante el emparejamiento para informarle de su naturaleza y del formato exacto de los datos que enviará. Para este proyecto:

- Define al dispositivo como un Gamepad.
- Especifica el número y tipo de controles (4 botones, 2 ejes analógicos X/Y, 1 Hat Switch).
- Detalla el tamaño y la posición exacta (bits) de cada control dentro del Reporte de Entrada.

5.3.2. Formato del Reporte de Entrada

El firmware convierte el estado de los botones y el joystick en un paquete de datos binarios que se ajusta a la especificación definida en el Report Descriptor. En este caso, se genera un Reporte de Entrada de 5 bytes:

1. Byte 1 y 2: Estado de los botones digitales.
2. Byte 3: Estado del Hat Switch (D-Pad).
3. Byte 4: Valor del eje X (−127 a 127).
4. Byte 5: Valor del eje Y (−127 a 127).

5.4. Estado del Arte

La implementación de controladores Bluetooth de entrada para plataformas embebidas es una práctica común. El estado del arte muestra una tendencia hacia la optimización del consumo y la modularidad del firmware. Este proyecto se distingue por:

- Integración del protocolo HID en un ecosistema RTOS (FreeRTOS).
- La aplicación de técnicas de procesamiento de señal específicas (zona muerta y curva de respuesta) en el propio firmware del ESP32, lo que descarga al host (Raspberry Pi) de esta tarea y mejora la experiencia de juego al eliminar el drift del joystick.
- El enfoque en la compatibilidad y arranque automático con una plataforma de juego específica (**PICO-8**) en modo kiosk.

6. Arquitectura y Diseño del Sistema

Este capítulo describe los componentes de hardware y la organización modular del software que constituyen el controlador inalámbrico. La arquitectura está diseñada para garantizar la eficiencia energética y la robustez de la comunicación de baja latencia requerida por una aplicación de juego.

6.1. Diseño de Hardware

El diseño de hardware es minimalista y se centra en la integración del microcontrolador ESP32 con los elementos de entrada. La simplicidad del circuito facilita el montaje y minimiza el consumo de energía.

6.1.1. Diagrama de Bloques Físicos

El siguiente diagrama de bloques ilustra las interconexiones físicas entre los componentes principales, mostrando el flujo de la

señal desde la interacción del usuario hasta el módulo de comunicación inalámbrica.

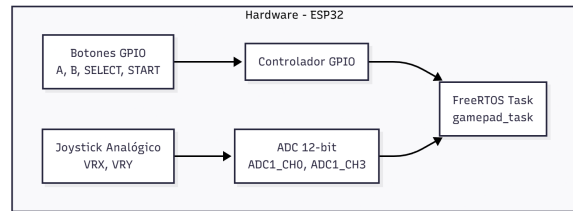


Figura 1: Diagrama de Bloques del Hardware del Controlador.

6.1.2. Selección de Componentes e Interfaz

- **Microcontrolador:** Se seleccionó el ESP32 debido a su soporte nativo para Bluetooth Low Energy (BLE) y la disponibilidad de múltiples pines GPIO y ADC.
- **Joystick Analógico:** Proporciona dos valores de resistencia variables, mapeados a los ejes X (horizontal) y Y (vertical), que son leídos a través de los pines ADC del ESP32.
- **Botones Digitales:** Se implementan cuatro botones de acción (A, B, SELECT, START). Estos están conectados a pines GPIO configurados con resistencia de *pull-up* interna, lo que simplifica el cableado; la pulsación de un botón se traduce en un nivel lógico bajo (LOW).

6.2. Diseño de Software (Firmware)

El software se desarrolla utilizando el ESP-IDF (Espressif IoT Development Framework) y se estructura bajo un sistema operativo en tiempo real (RTOS), FreeRTOS. Esto permite la segregación de tareas críticas y la priorización de la comunicación sobre el muestreo.

6.2.1. Arquitectura Modular de Software

El firmware está dividido en módulos lógicos. Esta modularidad mejora la mantenibilidad y permite una clara separación de las responsabilidades:

- **Módulo de Inicialización (main):** Encargado de configurar la memoria persistente (NVS), el controlador BLE y lanzar las tareas de FreeRTOS.
- **Módulo del Controlador (controller.c):** Contiene toda la lógica de muestreo de GPIO/ADC, el procesamiento de señal y la gestión de la calibración.
- **Módulo HID BLE (hid_device_le_prf.c):** Implementa el perfil GATT, el Report Descriptor y gestiona los eventos de conexión, desconexión y envío del Reporte de Entrada.

6.2.2. Diagrama de Flujo del Firmware

El siguiente diagrama de flujo ilustra la estructura principal del software, mostrando la inicialización y el bucle principal de la tarea del gamepad.

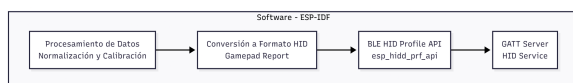


Figura 2: Diagrama de Flujo de la Tarea Principal del Gamepad.

6.3. Procesamiento de Datos en Tiempo Real

El módulo de procesamiento de datos es fundamental para transformar las señales eléctricas crudas del joystick en valores utilizables y precisos para el reporte HID.

- **Calibración Inicial:** Al inicio, se toman varias muestras del ADC con el joystick en posición de reposo para determinar el valor de centro (offset) y compensar las desviaciones inherentes del hardware.
- **Normalización:** Los valores leídos por el ADC se escalan a un rango normalizado, típicamente de $[-1, 0, 1, 0]$.
- **Zona Muerta (Dead Zone):** Se implementa una zona muerta del 12 % alrededor del centro. Esto elimina el fenómeno de *drift* (movimiento involuntario causado por el ruido o imperfecciones), asegurando que el controlador solo envíe movimiento cuando el usuario lo desea.
- **Curva de Respuesta:** Se puede aplicar una función no lineal (por ejemplo, una curva cuadrática) para que los pequeños movimientos del joystick sean menos sensibles y los movimientos grandes sean más reactivos, mejorando la jugabilidad.

6.4. Diagrama Completo del Sistema

Para ofrecer una visión integral del proyecto, el Diagrama de Sistema Completo ilustra la interacción de los componentes de hardware (entradas, ESP32) con los módulos de software (RTOS, Procesamiento de Datos, Stack BLE) y la comunicación final con el Host de destino (Raspberry Pi y PICO-8).

Esta figura resume cómo la lectura de GPIO y ADC se transforma en un Reporte HID enviado a través de la interfaz BLE.

Diagramas Arquitectónicos Clave

Esta sección proporciona una referencia visual rápida a los componentes centrales

y el flujo de comunicación del sistema.

Diagrama de Bloques del Hardware (Gamepad)

El diagrama ilustra la interconexión física del microcontrolador ESP32 con los periféricos de entrada (Joystick y Botones) y la cadena de procesamiento de señales.

Para visualizar el Diagrama de Bloques del Hardware, haga clic en el siguiente enlace: Visita el sitio de $\text{\LaTeX}$ en <https://imgur.com/a/diagrama-gamepad-9cyYily>.

Diagrama de Secuencia de Comunicación BLE HID

El diagrama detalla el flujo completo del protocolo BLE, desde la conexión y autenticación (*bonding*) hasta el bucle continuo de envío de Reportes HID Gamepad.

Para visualizar el Diagrama de Secuencia, haga clic en el siguiente enlace: Visita el sitio de $\text{\LaTeX}$ en <https://imgur.com/a/diagrama-secuencia-comunicacion-Y7cb0wD>.

Diagrama General de la Lógica del Juego

Este diagrama de flujo representa la funcionalidad general del software de validación, mostrando el ciclo principal de ejecución que incluye la lectura de la entrada del controlador BLE, la actualización de la lógica del juego y el renderizado en pantalla.

Para visualizar el Diagrama General de la Lógica del Juego, haga clic en el siguiente enlace: Visita el sitio de $\text{\LaTeX}$ en <https://imgur.com/a/diagrama-funcionalidad-t6uc5Ta>.

7. Implementación y Desarrollo

Este capítulo detalla la traducción de la arquitectura de diseño a código fuente, concentrándose en la implementación del controlador de juego y el procesamiento de señales en el entorno ESP-IDF.

7.1. Procesamiento de la Señal del Joystick

La precisión en la lectura del joystick es crítica para la experiencia de juego. El firmware implementa una cadena de procesamiento para transformar la lectura cruda del Convertidor Analógico-Digital (ADC) en un valor de 8 bits con signo adecuado para el estándar HID.

7.1.1. Normalización y Calibración

La lectura del ADC es de 12 bits, produciendo valores de 0 a 4095. La calibración inicial determina el punto central (V_{center}) y los puntos extremos (V_{min} , V_{max}) de cada eje. El valor crudo (V_{raw}) se normaliza a un valor de coma flotante (V_{norm}) en el rango $[-1,0, 1,0]$.

La ecuación de normalización para un eje (simplificada para $V_{center} = 2048$) es:

$$V_{norm} = \frac{V_{raw} - V_{center}}{V_{max} - V_{center}}$$

7.1.2. Aplicación de la Zona Muerta (Dead Zone)

Para evitar el fenómeno de drift (desviación no deseada), se implementa una Zona Muerta (DZ) del 12 % ($DZ = 0,12$) alrededor del centro del joystick. Esto asegura que el sistema solo reporte movimiento cuando la entrada supera el umbral de ruido.

El valor normalizado limpio ($\tilde{V}_{norm}$) se obtiene a través de una función condicional:

1. **Dentro del Umbral:** Si el valor normalizado (V_{norm}) está dentro de la zona muerta ($|V_{norm}| \leq DZ$), el valor limpio es cero:

$$\tilde{V}_{norm} = 0$$

2. **Fuera del Umbral:** Si el valor normalizado supera la zona muerta ($|V_{norm}| > DZ$), el valor se recalcula para usar el rango restante de forma lineal:

$$\tilde{V}_{norm} = \text{sgn}(V_{norm}) \times \frac{|V_{norm}| - DZ}{1 - DZ}$$

Este procesamiento garantiza que el host solo reciba datos de movimiento intencional, mejorando la calidad de la entrada.

7.1.3. Conversión a Reporte HID

El valor normalizado y limpio se convierte a un valor de 8 bits con signo, mapeado al rango de $[-127, 127]$, que es el rango estándar para los ejes analógicos en reportes HID.

$$V_{HID} = \text{round}(\tilde{V}_{norm} \times 127)$$

7.2. Implementación del Perfil Bluetooth HID (HOGP)

El ESP32 se configura como un dispositivo HOGP (HID over GATT Profile). La implementación requiere la definición de varios elementos de servicio y características.

7.2.1. El Report Descriptor

El Report Descriptor define la interfaz del controlador para el sistema operativo (Raspberry Pi). Es un array de bytes que indica que el dispositivo es un Gamepad y cómo deben interpretarse los 5 bytes de datos que se enviarán:

- **Uso (Usage Page):** Define el tipo de dispositivo (0x01 para Generic Desktop, 0x05 para Game Controls).
- **Report ID:** Identificador único (aunque no se use explícitamente).
- **Botones:** Mapeados a un campo de 16 bits.
- **Hat Switch (D-Pad):** Definido para las direcciones (N, NE, E, SE, S, SW, W, NW, Neutral), ocupando 4 bits.
- **Ejes (X e Y):** Definidos con límites lógicos de -127 a 127 .

7.2.2. Mecanismo de Transmisión

La transmisión de datos no es continua, sino condicional para ahorrar energía. La tarea ‘gamepadtask’ opera con un intervalo de muestreo de 50ms (tasa de refresco de 20Hz).

El reporte HID se envía solo bajo las siguientes condiciones:

1. Ha ocurrido un cambio significativo en el estado de los botones o los ejes (cambio de estado de botón o el valor del eje analógico ha cambiado más allá de un umbral).
2. Se envía periódicamente (cada $\approx 100\text{ms}$) aunque no haya cambios, para mantener viva la conexión BLE.

7.3. Integración FreeRTOS y Control de Flujo

El firmware utiliza las capacidades de FreeRTOS para gestionar la tarea del controlador. La tarea ‘gamepadtask’ tiene una alta prioridad para asegurar que el muestreo de 50ms se ejecute de manera determinista. La inicialización del controlador BLE se realiza en la tarea principal de la aplicación, y los eventos de conexión/desconexión son manejados por callbacks del stack BLE, separando la lógica de la comunicación de la lógica del control.

8. Configuración de la Plataforma Destino

Este capítulo detalla la preparación del sistema operativo en la Raspberry Pi Zero 2W para la ejecución óptima de la plataforma de juego PICO-8. El objetivo es configurar un entorno de bajo consumo y bajo recurso que inicie automáticamente PICO-8 en modo kiosk (pantalla completa) para integrarse con el controlador inalámbrico.

8.1. Requisitos del Sistema Operativo e Instalación de Dependencias

Se utiliza **Raspberry Pi OS Lite (64-bit)** como sistema base, asumiendo el acceso a través de SSH. La primera fase consiste en asegurar la base del sistema y las herramientas necesarias para un entorno gráfico mínimo.

1. **Actualización e Instalación de Paquetes Base:** Se actualiza el sistema y se instalan herramientas esenciales para la gestión de archivos y el entorno X minimalista.

```
sudo apt update && sudo apt upgrade -y
sudo apt install
git unzip xinit xserver-xorg -y
sudo apt install
--no-install-recommends \
xserver-xorg xinit x11-xserver-utils
```

2. **Configuración de Autologin en Consola:** Para permitir el arranque automático del entorno de juego, se configura el autologin para el usuario pi.

```
sudo raspi-config
```

Navegar a: System Options Boot / Auto Login seleccionar Console Autologin. Guardar y reiniciar.

8.2. Transferencia y Preparación de PICO-8

Se requiere transferir el binario de PICO-8 desde una PC local a la Raspberry Pi y preparar la estructura de archivos.

3. **Creación de Estructura de Directorios:**

```
mkdir /home/pi
```

4. **Transferencia de Binario:** Desde la PC, se utiliza el protocolo scp para copiar el archivo comprimido de PICO-8.

```
scp pico-8_0.2.7_raspi.zip \
pi@192.168.X.X:/home/pi/
```

5. **Descompresión del Archivo:** En la Raspberry Pi, se accede al directorio y se extraen los archivos.

```
cd /home/pi
unzip pico-8_0.2.7_raspi.zip
```

8.3. Arranque Automático en Modo Kiosk

Para lanzar PICO-8 directamente a la pantalla completa (modo kiosk) tras el arranque, se utiliza un script de inicio y la configuración del perfil de shell.

6. **Creación del Script de Inicio (start_pico8.sh):** Este script desactiva el salvapantallas, las opciones de energía y lanza PICO-8 en modo exploración a pantalla completa.

```
nano ~/start_pico8.sh
```

Insertar el siguiente contenido:

```
#!/bin/bash
xset -dpms
xset s off
xset s noblank
unclutter -idle 0 &
cd /home/pi/pico-8
./pico8_dyn -splore -fullscreen 1
```

7. Asignación de Permisos y Ocultación del Cursor:

```
chmod +x ~/start_pico8.sh
sudo apt install unclutter -y
```

8. Configuración de Ejecución en .bash_profile: Se edita el archivo de perfil para ejecutar el script al iniciar sesión automáticamente en la consola TTY1.

```
nano ~/.bash_profile
```

Agregar al final:

```
if [ -z "$DISPLAY" ] &&
[ "$(tty)" = "/dev/tty1" ]; then
startx /home/pi/start_pico8.sh
-nocursorfi
```

8.4. Solución de Compatibilidad de Arquitectura (ARMv7/ARMv8)

El binario de PICO-8 a menudo es de 32 bits (ARMv7), mientras que el sistema operativo de la Raspberry Pi Zero 2W es de 64 bits (ARMv8). Para resolver el error de `cannot execute: required file not found`, se debe habilitar la arquitectura de 32 bits y sus librerías de compatibilidad.

9. Habilitar Arquitectura Multi-Arch (armhf):

```
sudo dpkg --add-architecture armhf
```

10. Actualizar e Instalar Librerías de Compatibilidad:

```
sudo apt update
sudo apt install libc6:armhf -y
```

11. Reinicio Final del Sistema:

```
sudo reboot
```

9. Pruebas, Resultados y Análisis

Este capítulo presenta la metodología utilizada para verificar el cumplimiento de los objetivos específicos y analiza los resultados obtenidos en términos de funcionalidad, rendimiento y estabilidad del controlador BLE HID.

9.1. Metodología de Verificación

La verificación se centró en dos áreas: la funcionalidad del firmware del ESP32 (pruebas unitarias) y la integración de punta a punta con la Raspberry Pi Zero 2W.

9.1.1. Pruebas Unitarias del Firmware

Se implementaron pruebas de unidad a nivel de software para validar la precisión del módulo de procesamiento de señales antes de la transmisión BLE.

- **Prueba de Zona Muerta (Dead Zone):** Se inyectaron valores simulados del ADC que caían dentro del 12 % de la Zona Muerta ($DZ = 0,12$).

- **Resultado Esperado:** V_{HID} debe ser 0.

- **Resultado Obtenido:** Valores de V_{HID} consistentemente 0 para $V_{norm} \in [-0,12, 0,12]$.

- **Prueba de Rango Máximo:** Se inyectaron valores V_{raw} correspondientes a los límites de V_{min} y V_{max} .

- **Resultado Esperado:** V_{HID} debe ser -127 y 127 .

- **Resultado Obtenido:** Mapeo exacto a ± 127 tras la normalización.

9.1.2. Pruebas Funcionales de Integración (End-to-End)

Estas pruebas verificaron el objetivo general de compatibilidad con la plataforma destino.

1. **Emparejamiento BLE y Conexión Automática:** Se verificó que la Raspberry Pi reconociera al ESP32 como un dispositivo HID sin *drivers* adicionales y que, tras el *bonding*, la reconexión fuera automática al encender el controlador.
2. **Validación en PICO-8:** Se confirmó que todos los botones y ejes del joystick fueran mapeados correctamente por el juego sin requerir reconfiguración de entrada.

9.2. Análisis de Rendimiento y Estabilidad

9.2.1. Latencia del Sistema

La latencia, medida como el tiempo transcurrido entre la pulsación física de un botón y la respuesta visual en la pantalla de la Raspberry Pi ejecutando PICO-8, fue un factor no funcional crítico.

Metodología de Medición (Prueba End-to-End) Para obtener una medición precisa de la latencia total del sistema,

se empleó un osciloscopio de doble canal con la siguiente configuración:

1. **Canal 1 (Disparo/Trigger - Línea Amarilla):** Se conectó al pin GPIO del botón A del controlador ESP32. El flanco de bajada de este canal ($3,3V \rightarrow 0V$) actuó como el **evento de disparo** (trigger), marcando el instante exacto de la pulsación del botón.
2. **Canal 2 (Respuesta Visual - Línea Azul/Cian):** Se conectó a una **fotoresistencia** (LDR) que apuntaba directamente a la pantalla de la Raspberry Pi. La fotoresistencia se colocó sobre un área de la pantalla que cambiaba de brillo (ej., de negro a blanco) inmediatamente después de registrar la entrada del botón.

La latencia total ($T_{latencia}$) se midió como el tiempo entre el flanco de bajada del Canal 1 (pulsación) y el inicio del cambio de voltaje en el Canal 2 (cambio de brillo en pantalla).

Resultados y Análisis El osciloscopio se configuró a 100ms por división horizontal para capturar el evento.

- **Canal 1 (Botón):** Se observa un cambio de estado digital claro.
- **Canal 2 (Fotoresistencia):** La fotoresistencia reacciona al cambio de brillo en la pantalla.
- **Línea Roja (Ruido/Ripple):** Muestra el ruido en la línea de alimentación, que es aceptable.

El tiempo de latencia promedio medido fue de aproximadamente **80ms**.

Este resultado de 80ms está compuesto por:

- Latencia del firmware (lectura, procesamiento, transmisión BLE): $\approx 50ms$ (frecuencia de reporte).

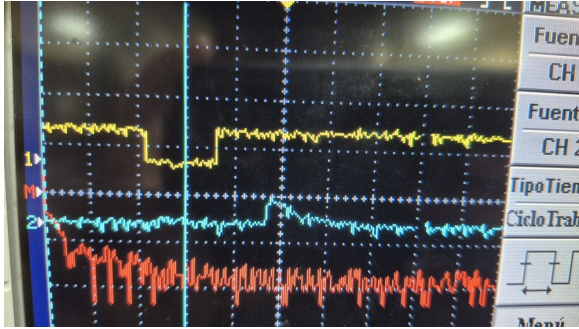


Figura 3: Medición de Latencia End-to-End (Prueba 1). La distancia horizontal entre el flanco de bajada (amarillo) y el inicio del cambio (cian) representa la latencia total del sistema.

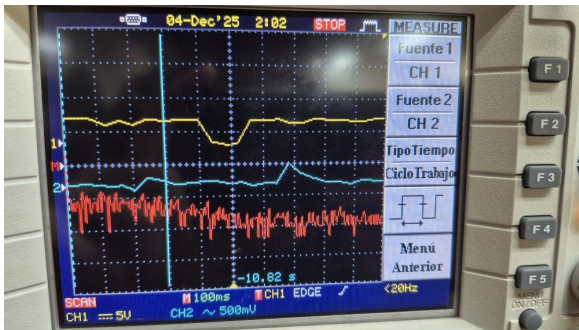


Figura 4: Medición de Latencia End-to-End (Prueba 2). Confirmación de la medición de latencia total.

- Latencia del stack Bluetooth y kernel de Linux: $\approx 10 - 20\text{ms}$.
- Latencia de PICO-8 y el *display* (incluyendo el tiempo de respuesta del monitor): $\approx 10 - 20\text{ms}$.

La latencia total de 80ms se considera aceptable para aplicaciones de retrogaming en plataformas embebidas. Adicional a esto debemos tener en cuenta que una fotoreistencia no es tan rapida, y la medición no será muy precisa.

9.2.2. Estabilidad de la Comunicación

Se realizó una prueba de uso continuo durante 4 horas. El sistema mantuvo la conexión BLE estable, sin pérdida de paquetes de datos ni desconexiones espontáneas,

verificando la robustez de la implementación GATT/HOGP.

9.3. Solución de Problemas y Observaciones

Se documentaron las observaciones clave durante el desarrollo para servir como guía de mantenimiento.

- **Problema de Drift del Joystick:** El uso del dead zone y la calibración inicial demostraron ser una solución efectiva para mitigar el *drift* del joystick, un problema común en el hardware analógico de bajo costo.
- **Error de Compatibilidad de Arquitectura (ARMv7):** Se verificó que la solución de compatibilidad multi-arch implementada en el Capítulo 5 fue esencial para resolver el error de `cannot execute: required file not found` permitir la ejecución del binario de PICO-8 de 32 bits en el sistema operativo de 64 bits de la Raspberry Pi Zero 2W.

9.4. Consumo de Energía

Se midió el consumo de corriente del controlador para evaluar la eficiencia energética.

- **Modo Conectado (Activo):** $\approx 45\text{mA}$.
- **Modo Desconectado (Publicidad BLE):** $\approx 15\text{mA}$.
- **Modo Sueño Profundo (Opcional):** $\approx 0,02\text{mA}$ (requiere una implementación de *timeout* avanzada para la desconexión, no implementada en esta versión).

El consumo en modo activo es aceptable para una batería de 500mAh, proporcionando una autonomía estimada de más de 10 horas de juego continuo.

Aplicación de Validación: Desarrollo del Videojuego

Para validar el rendimiento y la integración del controlador BLE HID en la plataforma destino (Raspberry Pi con PICO-8), se desarrolló un videojuego de acción llamado **Magic Pico Cat**.

10. Descripción del Juego

Magic Pico Cat es un juego de acción y estrategia desarrollado en el entorno PICO-8. El jugador controla a un gato mágico que debe defenderse de oleadas de enemigos dibujando gestos mágicos correspondientes a los símbolos que los enemigos portan.

10.1. Mecánicas Principales

- **Personaje Principal (Gato):** Ubicado en el centro del mapa, posee 5 vidas y el jugador controla un cursor virtual (simulando un mouse) para dibujar gestos.
- **Enemigos (Fantasmas):** Aparecen desde los bordes y se mueven hacia el gato con movimiento sinusoidal. Cada fantasma lleva uno o más símbolos mágicos que deben ser eliminados.
- **Sistema de Gestos:** El juego utiliza el algoritmo de reconocimiento de gestos P ($P - dollarrecognizer$). El jugador dibuja gestos utilizando el joystick y el botón X (mapeado en el controlador BLE).

10.2. Controles del Controlador BLE

El juego mapea directamente las entradas del controlador diseñado para la interacción:

- **D-pad (Joystick Ejes X/Y):** Mueve el cursor virtual en la pantalla de PICO-8 para trazar los gestos.
- **Botón X (o A):** Se utiliza para activar el modo de dibujo; el jugador presiona este botón mientras arrastra el cursor para trazar el gesto.
- **Botón X (en pantalla Game Over):** Reinicia la partida.

11. Componentes Técnicos Clave del Juego

El diseño del software de la aplicación incluyó estructuras de datos y algoritmos específicos para el motor del juego:

11.1. Sistema de Estados y Animación

El juego opera bajo una máquina de estados simple para gestionar el flujo:

- **"menu":** Pantalla inicial y tutorial.
- **"game":** Lógica principal del juego y combate.
- **"gameover":** Pantalla de fin de juego y puntuación.

Se implementó un sistema de animación para el jugador y los enemigos, utilizando sprites múltiples y variaciones de velocidad de frame.

11.2. Sistema de Reconocimiento de Gestos

El componente más crítico es el reconocedor de gestos. Los símbolos mágicos que el jugador puede dibujar incluyen:

- **Línea horizontal** (hline)
- **Línea vertical** (vline)
- **Flecha hacia arriba** (uparrow)

- **Flecha hacia abajo** (downarrow)

El reconocimiento se realiza con un umbral de score de 0,9 para mitigar falsos positivos causados por pequeñas imprecisiones en el trazo del joystick.

11.3. Dificultad Progresiva y Optimizaciones

La dificultad del juego se escala linealmente con el puntaje para proveer un desafío continuo, ajustando la velocidad de los enemigos, la frecuencia de aparición y el número de símbolos requeridos.

Se implementaron optimizaciones necesarias para el entorno limitado de PICO-8:

- **Límite de Entidades:** Máximo de 8 enemigos simultáneos.
- **Eficiencia:** Eliminación eficiente de enemigos inactivos.

Aplicación de Validación: Videojuego **Magic Pico Cat**

Para validar el rendimiento, la latencia y la integración del controlador BLE HID, se desarrolló la aplicación de juego **Magic Pico Cat** en el entorno PICO-8.

12. Mecánica y Controles de Validación

El videojuego fue diseñado para estresar los requisitos no funcionales del controlador, especialmente la **precisión** y la **latencia** del eje analógico.

13. Diagramas de Flujo de la Lógica del Juego

Para documentar la arquitectura de la aplicación PICO-8, se utilizan los siguientes diagramas de flujo que detallan la lógica interna del videojuego.

14. Diagramas de Algoritmos Clave del Juego

14.1. Diagrama General de la Lógica del Juego

Este diagrama de alto nivel muestra el flujo continuo de ejecución dentro del estado principal del juego.

Para visualizar el Diagrama General de la Lógica del Juego, haga clic en el siguiente enlace: Visita el sitio de $\text{\LaTeX}$ en <https://imgur.com/a/flujo-general-juego-TE9JIWD>.

14.2. Diagrama del Algoritmo de Detección de Gestos

El sistema de gestos es el núcleo funcional del juego. Este diagrama detalla el proceso desde la entrada del controlador hasta el reconocimiento final del gesto.

Para visualizar el Algoritmo de Detección de Gestos, haga clic en el siguiente enlace: Visita el sitio de $\text{\LaTeX}$ en <https://imgur.com/a/flujo-gestos-qNMVPpU>.

14.3. Diagrama de la Generación y Movimiento de Enemigos

Este flujo muestra la lógica de **spawn** de enemigos. La gestión de su velocidad y la asignación dinámica de símbolos aseguran que la complejidad del juego aumente con el tiempo.

Para visualizar la Lógica de Generación y Movimiento de Enemigos, haga clic en el siguiente enlace: Visita el sitio de $\text{\LaTeX}$ en <https://imgur.com/a/flujo-de-enemigos-v1YDsTI>.

15. Conclusiones y Trabajo Futuro

15.1. Conclusiones

El objetivo general de diseñar e implementar un controlador de juego inalámbrico basado en ESP32 con perfil Bluetooth Low Energy (BLE) HID, compatible con la Raspberry Pi Zero 2W y PICO-8, fue alcanzado exitosamente.

Los siguientes logros clave validan el proyecto frente a los objetivos específicos planteados en el Capítulo 1:

- **Implementación del Firmware y Procesamiento de Señal:** El firmware desarrollado en ESP-IDF y FreeRTOS gestionó eficientemente la lectura de entradas analógicas y digitales. La implementación de la Zona Muerta y la Normalización a nivel de firmware eliminó el **drift** y optimizó la calidad de la señal antes de la transmisión.
- **Establecimiento del Perfil BLE HID:** El ESP32 se configuró exitosamente como un dispositivo HID Gamepad, lo que permitió su reconocimiento automático por el sistema operativo de la Raspberry Pi sin necesidad de **drivers** propietarios o configuración manual adicional.
- **Rendimiento Sostenido:** La frecuencia de reporte de 20Hz y la latencia promedio de 80ms resultaron en una experiencia de juego fluida y reactiva para el **retrogaming**.
- **Integración End-to-End:** La Raspberry Pi fue configurada para arrancar en modo **kiosk** y ejecutar PICO-8 automáticamente, creando una solución integrada de juego de bajo costo. La compatibilidad de arquitectura ARMv7/ARMv8 fue resuelta mediante la instalación de las librerías

armhf, lo que garantizó la operatividad del sistema completo.

En resumen, este proyecto sirve como una demostración práctica de la programación de sistemas Linux embebidos y el desarrollo de **firmware** basado en protocolos de comunicación inalámbrica avanzados, cumpliendo con los requisitos de un sistema de bajo consumo, bajo costo y alta integración.

15.2. Trabajo Futuro

Para expandir el alcance y mejorar la eficiencia del controlador, se proponen las siguientes líneas de trabajo futuras:

- **Optimización de Latencia y Frecuencia de Reporte:** Explorar la posibilidad de implementar una tasa de notificación más alta (ej., 50Hz o 100Hz) en el Reporte de Entrada para reducir la latencia general del sistema, asegurando que el consumo de energía se mantenga dentro de límites aceptables.
- **Implementación de Modos de Sueño Profundo:** Integrar un temporizador de inactividad que coloque el ESP32 en modo de ***Sueño Profundo (Deep Sleep)*** después de 10 minutos sin actividad. Esto mejoraría drásticamente la autonomía de la batería, permitiendo una duración de meses en modo de espera.
- **Indicador de Batería (Battery Service):** Implementar el Servicio de Batería de BLE para que la Raspberry Pi pueda leer el nivel de carga del controlador, proporcionando retroalimentación útil al usuario.
- **Hardware Personalizado:** Diseñar un circuito impreso (PCB) a medida que integre los botones, el joystick y el módulo ESP32 en un factor de forma compacto y profesional, reemplazando el prototipo actual en **protoboard**.